Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Ders büyük olasılıkla 12 Nisan 2023 Çarşamba günü online yapıldı (tarih altyazıda geçmiyor). Vizeden önceki son konu dersiydi. Dersin büyük kısmı vize öncesi tekrar.
- Ders planındaki değişiklikler ve sınav düzeni duyuruldu.
- Tekrar edilen konular:
  - CASE (Computer-Aided Software Engineering) araçları.
  - Yazılım süreç modelleri: şelale, prototip, RAD, artımlı, spiral, Unified Process / RUP, çevik (agile) modeller (XP, Scrum, Kanban).
  - CMMI süreç olgunluk sertifikasyonu ve düzeyleri.
  - Gereksinim mühendisliği ve SRS belgesi.
- IEEE 830 yapısına göre hazırlanmış örnek bir SRS belgesi incelendi: "Etkinlik Duyuru Sistemi" projesi.
- Etkinlik şemasında (activity diagram) kulvarlı iki örnek: teklif (kota) hazırlama ve sipariş alma.
- Tasarım kalıplarının tekrarı.
- Yeni konu: yazılım kalitesi.
  - McCall'ın kalite faktörleri ve üç bakış açısı.
  - Yeniden kullanılabilirliği sağlayan yapılar.
  - Hata bulma maliyetinin süreç boyunca artması.
  - Kaliteyi sağlama adımları.
  - Test türlerine kısa giriş.

Hocanın Özellikle Vurguladığı Kısımlar

- **Sınav düzeni değişti:** Tek vize yapılacak. Vize 9. haftada online olacak. Önceden iki vize (ya da vize ve kısa sınav) planlanmıştı. Vize ile proje notunun ağırlıkları sonra duyurulacak.
- **Kalite özellikleri vizeye dahil.** Test teknikleri vizeye dahil değil, vizeden sonra işlenecek.
- **Hataları erken bulun.** Süreç ilerledikçe hata bulma maliyeti artar, hata bulma olasılığı düşer. Her kilometre taşında gözden geçirme (review) yapılmalıdır.
- **Projeye başlamadan bir süreç modeli seçin ve ona uyun.** Modelin kendi kontrolleri işinizi kolaylaştırır.
- **SRS belgesinde her işlev bloğu için bir kullanım öyküsü belgesi** bulunmalıdır.
- **Etkinlik şemasının asıl faydası**, hangi kullanıcının hangi eylemi yaptığını kulvarlarla ayrı ayrı göstermesidir. Bu, kullanım şemasından farkıdır.

Kısa Tekrar Notları

- **CASE araçları:** Yazılım geliştirmenin her evresinde geliştiricilere destek olan yazılımlar. UML şemalarını hızlı çizer ve kontrol eder. Kodlama, konfigürasyon yönetimi ve test aşamalarında da kullanılır. Bütünleşik büyük araçlar ve tek aşamaya yönelik küçük araçlar vardır.
- **Süreç modelleri:** şelale (waterfall), prototip (ön ürün), RAD (Rapid Application Development), bileşen tabanlı, artımlı (incremental), spiral, Unified Process, çevik (XP, Scrum, Kanban).

- **CMMI:** Capability Maturity Model Integration, yani bütünleşik yetkinlik olgunluk modeli. Yazılım geliştirme birimini olgunluk düzeyine göre derecelendirir.
- **SRS:** Software Requirements Specification. Gereksinim analizinde yapılan çalışmaların belgesidir.
- **IEEE 830-1998:** SRS belgesi için önerilen içerik yapısı.
- **McCall bakış açıları (hocanın adlandırması):** kullanıcı (ürünün işletimi), değişiklik/bakım yapan (ürünün gözden geçirilmesi), başka ortama taşıyan (ürünün geçişi). Hoca bazı özellikleri farklı gruba koydu; McCall'daki gruplar için bkz. ders sonrası notlar.
- **Kalite özellikleri:** doğruluk (correctness), güvenilirlik (reliability), verimlilik (efficiency), güvenlik (integrity/security), kullanışlılık (usability), bakım kolaylığı (maintainability), esneklik (flexibility), test edilebilirlik (testability), taşınabilirlik (portability), yeniden kullanılabilirlik (reusability), birlikte çalışabilirlik (interoperability).
- **Yeniden kullanılabilirliği artıranlar:** modülerlik, az bağımlılık (low coupling), yüksek uyum (high cohesion), tasarım kalıpları, yazılım kütüphaneleri, nesneye yönelik yaklaşım.
- **Test türleri:** saydam kutu (white box), kara kutu (black box), gri kutu (gray box).
- **Test adımları:** birim testi, bütünleştirme testi, sistem testi, kullanıcı kabul testi (user acceptance test).

Detaylı Açıklamalar

CASE araçları

Bir öğrenci CASE araçlarını "UML şemalarını program yardımıyla çizmek için kullanılan programlar" diye tanımladı. Hoca bunun yalnız analiz ve tasarım desteği olduğunu söyledi. CASE araçları kodlamada, konfigürasyon yönetiminde ve testte (birim, bütünleştirme, sistem, kullanıcı testi) de kullanılır. Açık kaynaklı CASE araçları vardır. Büyük projelerde kullanılan profesyonel araçlar ise çok pahalıdır.

Süreç modelleri ve RUP

Klasik modellerden şelale, prototip, RAD, artımlı ve spiral model işlendi. Hoca artımlı yaklaşımı spiral modelin artımlı yapısının ilk örneği olarak tanıttı. Unified Process modeli, IBM'in sonradan satın aldığı Rational firmasının RUP (Rational Unified Process) yaklaşımına dayanır. Büyük firmalar bu modeli çok kullanır. Çevik modeller önce küçük ve orta boy şirketlerde yaygınlaştı. Bugün büyük askerî yazılımlarda da kullanılıyor. Derste çevik modellerden XP ve Scrum işlendi.

CMMI düzeyleri

Hoca CMMI düzeylerini şöyle anlattı:

- **2. düzey:** Projelerde gereksinim analizi yapılır ve SRS belgesi hazırlanır. Tasarımın sonunda tasarım belgesi oluşturulur. Proje planı yapılır. Konfigürasyon yönetimi uygulanır.
- **Konfigürasyon yönetimi:** Yazılımın her aşamada ürettiği ürünlerin (belgeler, kod, sürümler) kaydının tutulmasıdır. Her sürümde hangi elemanın hangi durumda olduğu izlenir.
- **3. düzey:** Risk yönetimi yapılır. Bütün projelerde aynı yapıda gereksinim belgesi ve aynı belgeleme standardı kullanılır. Yani süreç şirket genelinde ortaktır.
- **Sonraki gelişmeler:** CMMI, 2000'lerin başında teknik süreçleri ve proje yönetimini birleştirdi; bu yüzden "bütünleşik" adını aldı. 2018'deki sürümle insan kaynağına ilişkin yetkinlikler de eklendi: geliştirici alımı, yeteneklerin kullanımı, kariyer planlaması.
- **Türkiye'deki örnekler:** Bir öğrenci Koç Sistem ve ETİYA'yı (altyazıda "İtia") saydı. Hoca askerî yazılım üreten Milsoft'un 5. düzeyde olduğunu, bu alanda yazılım üreten diğer firmaların en az 4. düzeyde bulunduğunu söyledi. Hocaya göre bu alanda yazılım üreten firmaların en az 3. düzeyde olması şartı vardır.

Örnek SRS belgesi: Etkinlik Duyuru Sistemi

Belge, IEEE 830 standardındaki içerik listesine göre hazırlanmış. Sistemde etkinlikler aranır, listelenir ve ayrıntıları görüntülenir. Bazı etkinlikler için satın alma yapılabilir. Bilgi girişini idari personel yapar.

Belgenin bölümleri:

- **1. Giriş:** amaç, kapsam, tanımlar ve kısaltmalar, referanslar, belgeye genel bakış.
- **2. Genel tanım:** ürüne bakış (blok şema), ürün işlevleri, kullanıcı, donanım ve yazılım arayüzleri.
- **3. Özel gereksinimler:**
  - İşlevsel gereksinimler, madde madde numaralanır: 1.1.1, 1.1.2 ... Örnekler: etkinlik ekleme, silme, güncelleme, listeleme.
  - Performans gereksinimleri.
  - Tasarım kısıtları.
  - Kalite özellikleri. Bunlar işlevsel olmayan gereksinimlerdir (non-functional requirements).
  - Gereksinimlerin izlenebilirliği.
- **Ekler:** ekran tasarımları, kullanım öyküsü şablonları, kullanım şemaları, ER şemaları.

Kullanım öyküsü belgesinde numara, ad, hazırlayan, aktörler, birincil aktör, ön koşul, son koşul, kullanım sıklığı ve önceliği bulunur. Ana akışta (bu örnekte 12 adım) sistem normal çalışır. Alternatif akış, herhangi bir adımda bir sorun çıkarsa ne olacağını ve nereye dönüleceğini gösterir. Hoca örnek belgede birincil aktörün açıklanmadığını eksik olarak belirtti.

Etkinlik şeması sembolleri

- **Başlangıç:** dolu daire. **Bitiş:** halka içinde dolu daire.
- **Kulvar başlığı:** işi hangi kullanıcı grubunun yaptığını gösterir.
- **Senkronizasyon çubuğu:** Çatal (fork/split) bir akışı iki paralel kola ayırır. Birleşme (join) iki kolu tekrar birleştirir.
- **Karar:** eşkenar dörtgen, üzerine karar koşulu yazılır.

Örnek 1: Teklif (kota) hazırlama

Kulvarlar: müşteri, satıcı, teknik uzman, sistem.

```
Müşteri:  teklif ister
Satıcı:   gereksinimleri not alır, "teknik destek gerekli mi?" diye sorar
   [evet] -> Teknik uzman: gereksinimleri değerlendirir, sisteme girer, teklifi hesaplar
   [hayır] -> Satıcı: sisteme girer, teklifi hesaplar
Müşteri:  teklifi gözden geçirir
   [değişiklik istiyor] -> en başa döner
   [istemiyor] -> siparişi kabul eder -> (son)
```

Örnek 2: Sipariş alma

```
Satış:       siparişi kabul et -> siparişi oluştur
Mühendislik: spesifikasyonları oluştur
   FORK
     Satın alma: malzemeleri satın al
     Üretim:     üretimi gerçekleştir
   JOIN
Planlama:    üretim planını (schedule) oluştur -> (son)
```

Etkinlik şeması, kullanım şemasından sonra, tasarıma geçerken çizilir. Kullanım şemasının ayrıntılandırılmış hâli gibidir. Bu aşamada eksik bilgi için kullanıcılara açık uçlu ve kapalı uçlu sorular ya da anketler yöneltilebilir.

Tasarım kalıplarının tekrarı

Tasarım kalıbı, daha önce çözülmüş ve verimliliği kanıtlanmış bir çözümün, aynı problemle karşılaşanlar için hazır tutulmasıdır. Yeniden kullanılabilirliği sağlayan en önemli etkenlerden biridir. Hoca geçen hafta işlenen dokuz kalıbı tekrar saydı: yaratıcı (Creator), bilginin uzmanı (Information Expert), az bağımlılık (Low Coupling), denetçi (Controller), yüksek uyum (High Cohesion), çok şekillilik (Polymorphism), yapay sınıf (Pure Fabrication), dolaylılık (Indirection), değişimlerden korunma (Protected Variations). Ayrıntılar için Ders 6 özetine bakın.

Yazılım kalitesi: McCall modeli

Hoca kalite özelliklerini McCall'ın bakış açılarına göre anlattı:

- **Doğruluk:** Yazılım SRS'deki spesifikasyonlara uyuyor ve müşteri isteklerini karşılıyor mu?
- **Güvenilirlik:** Tasarlanan işlevler istenen duyarlılıkla yerine getiriliyor mu? Örneğin bir muhasebe yazılımı doğru sonucu %99 mu, %99,8 mi güvenle veriyor? Güvenilirlik doğruluktan da güvenlikten de farklıdır.
- **Birlikte çalışabilirlik:** İki sistemin birlikte çalışabilmesi ve veri alışverişi yapabilmesi.
- **Verimlilik:** İşlevler için gereken bilgi işlem kaynakları ve kodlama gideri.
- **Güvenlik:** Yetkisiz kişilerin yazılıma erişiminin önlenmesi.
- **Kullanışlılık:** Öğrenme, kullanma, girdi hazırlama ve çıktı yorumlama kolaylığı.
- **Bakım tarafı:** hata bulma kolaylığı, esneklik, test etme kolaylığı.
- **Taşınabilirlik:** Programın farklı donanım ve yazılım ortamlarında kullanılabilmesi.
- **Yeniden kullanılabilirlik:** Programın tamamının ya da bir bölümünün başka bir uygulamada kullanılabilmesi.

ISO standartları benzer özellikleri bakış açılarına ayırmadan tek liste hâlinde verir.

Yeniden kullanılabilirlik nasıl sağlanır?

Öğrenci önerileri:

- İş kodunu (core) ayrı tutmak.
- Veritabanı ekleme/silme kodunu, loglamayı ve kimlik doğrulamayı bağımsız modüllere ayırmak.

Hocanın eklemeleri:

- Tasarımda az bağımlılık ve yüksek uyum. Modüller ve sınıflar ne kadar bağımsız olursa yeniden kullanım o kadar artar.
- Tasarım kalıpları. Bir kez çözülen problemin çözümü defalarca kullanılır.
- Yazılım kütüphaneleri. Genelleştirilmiş fonksiyonlar şirket içinde ya da bir dil ortamında herkesin kullanımına açılır.
- Nesneye yönelik yaklaşımın kendisi.

Kaliteyi sağlamak

Süreç boyunca yapılması gerekenler:

- **Gereksinim analizinde:** sistemin kolay öğrenilir olmasına çaba gösterilir.
- **Modüllere ayırırken:** taşınabilirlik ve birlikte çalışabilirliğe önem verilir.
- **Ayrıntılı tasarımda:** doğruluğa ağırlık verilir; algoritmaların istenenleri adım adım karşıladığı kontrol edilir.
- **Testte:** doğruluk ve performans ölçülür.

Hata bulma maliyeti süreç ilerledikçe artar. Hata bulma olasılığı ise azalır. Bu yüzden hatalar olabildiğince erken bulunmalıdır. Bunun için:

- Her kilometre taşında (milestone) gözden geçirme (review) yapılır.
- Analiz belgeleri, tasarım belgeleri ve kod çalıştırılmadan incelenir (code inspection).
- Kodlamaya geçildikçe birim testleri yapılır.
- Geliştirmeye başlamadan bir süreç modeli seçilir ve modele uyulur.

Kalite güvencesi adımları:

- Geliştirmenin planlanmasında kalite kontrol yöntemlerinin ve araçlarının belirlenmesi.
- Durak noktalarında gözden geçirme.
- Kaynak programın sınanması.
- Konfigürasyon yönetimi: konfigürasyonun kurulması, değişikliklerin kontrolü, raporlama.

Test türlerine giriş

Hoca saydam kutu ve kara kutu testinin yanında üçüncü bir tür olarak gri kutu testini andı. Test süreci dört adımdan oluşur: birim testi, bütünleştirme testi, sistem testi ve kullanıcı kabul testi. Ayrıntılar vizeden sonra işlenecek.

Ders sonrası notlar

- **Kalıp sayısı ve tarihi.** Hoca bir yerde kalıpların ilk kez 26 tane tanımlandığını, sonra 23 dediğini söyledi. Ayrıca bunları 2000'lerin başına tarihledi. Doğrusu büyük olasılıkla: Gamma, Helm, Johnson ve Vlissides (Gang of Four) 1994'te *Design Patterns* kitabında **23** kalıp tanımladı.
- **Dokuz kalıp GoF değil, GRASP.** Derste tekrar edilen dokuz kalıp GoF'un 23 kalıbından seçilmiş değildir. Bunlar Craig Larman'ın GRASP (General Responsibility Assignment Software Patterns) ilkeleridir. GoF kalıpları Singleton, Factory Method, Observer, Adapter, Strategy gibi kalıplardır.
- **CMMI'nin açılımı.** Altyazıda "Capability Majority Model" geçiyor. Doğrusu **Capability Maturity Model Integration**. 2018 gelişmesi büyük olasılıkla CMMI V2.0 sürümüdür.
- **CMMI düzey numaraları.** Olgunluk düzeyleri büyük olasılıkla 1'den 5'e kadardır: 1 Başlangıç (Initial), 2 Yönetilen (Managed), 3 Tanımlı (Defined), 4 Nicel Olarak Yönetilen (Quantitatively Managed), 5 Optimize Eden (Optimizing). Bir öğrencinin andığı "0. düzey" olgunluk düzeyi değildir; yalnız süreç alanlarının yetkinlik düzeylerinde (capability levels, 0–3) vardır.
- **IEEE 830 güncel değil.** IEEE 830-1998'in yerini büyük olasılıkla 2011'de ISO/IEC/IEEE 29148 standardı aldı. Bu standart büyük olasılıkla 2018'de güncellendi. 830 yapısı ders ve proje belgelerinde hâlâ yaygın kullanılır.
- **Etkinlik şemasında eylem sembolü.** Hoca bir yerde eylemlerin elipsle gösterildiğini söyledi. UML'de eylemler kenarları yuvarlatılmış dikdörtgenle çizilir. Elips kullanım şemasında senaryo sembolüdür.
- **Birlikte çalışabilirliğin grubu.** Hoca birlikte çalışabilirliği kullanıcı açısından önemli özellikler arasında saydı. McCall modelinde birlikte çalışabilirlik, taşınabilirlik ve yeniden kullanılabilirlikle birlikte ürünün geçişi (product transition) grubundadır. Ürünün işletimi grubunda doğruluk, güvenilirlik, verimlilik, bütünlük ve kullanışlılık vardır. Ürünün gözden geçirilmesi grubunda ise bakım kolaylığı, esneklik ve test edilebilirlik vardır.
- **ISO kalite modeli.** Güncel yazılım kalite modeli büyük olasılıkla ISO/IEC 25010'dur. Bu model büyük olasılıkla 2011'de ISO/IEC 9126'nın yerini aldı.
- Altyazıda "bilim testi" diye geçen terim **birim testi**dir (unit test).

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.